MAPA DO PROJETO · OUTUBRO DE 2026 · REVISÃO 8 · PROPOSTA

IAEmPS2

Um mestre invisível por inteligência artificial para jogos de PlayStation 2 em emulador. Ele vê a tela, lê o estado da partida e intervém, primeiro no ritmo e depois no mundo do jogo, para a experiência deixar de ser a de sempre: alivia no perigo real, aperta quando o jogador domina, e muitas vezes não faz nada. Casos de Uso e Blueprint Técnico derivam deste documento.

Nome provisório: "IAEmPS2", o nome da pasta do protótipo. **Revisão 1, 26 de outubro de 2026:** fase 1 vira tela e alavancas do emulador; Final Fantasy XII sai; versão própria mínima do emulador; LC-02 e LC-03 fechadas. **Revisão 2, mesmo dia:** perfil de memória deixa de ser opcional e vira a fase 2, a que muda o mundo. **Revisão 3, mesmo dia:** seis decisões propostas aprovadas; LC-01 fechada. **Revisão 4, mesmo dia:** Blueprint Técnico aprovado; entram DC-15 a DC-19 e LC-11 a LC-13; disco deixa de ser risco. **Revisão 5, mesmo dia:** DC-11 revisada, repositório público, decisão do cliente para a integração contínua rodar sem custo. **Revisão 6, 27 de outubro de 2026:** o cliente quer que o mestre modifique o espaço 3D; entra a fase 4 (módulo K, camada de cena no emulador), com DC-20 e DC-21 propostas, DC-13 revisada, RN-11 e as lacunas LC-14 a LC-16; LC-10 fechada com número e LC-07 fechada para esta fatia; aprovada no mesmo dia. **Revisão 7, 27 de outubro de 2026:** a prova medida da cena (S7-00 e S7-01) mostrou que a identidade de um objeto não vem do desenho, e sim da memória do jogo; o cliente decidiu partir para a engenharia reversa do jogo. A fase 2 é adiantada (DC-22 proposta), com a câmera e o mapa de entidades (I01, antes na fase 3) na frente; DC-13 revisada de novo (projeto próprio, sem proposta ao original); LC-15 fechada; LC-17 aberta para o nível 3 (modelos e colisão), meta do cliente; LC-04 e LC-07 com resposta parcial para o jogo de abertura. **Revisão 8, 29 de outubro de 2026:** a fase 2 avançou do que estava previsto: vigia de memória, executável com símbolos, chamada remota de funções do jogo, personagens criados e módulos carregados pelo mestre a partir da ISO. O cliente decidiu mapear o jogo inteiro de forma sistemática antes das ações de mundo (DC-23 proposta); DC-22 fechada; LC-17 ganha caminho concreto; LC-18 aberta para a Ada de cena.

11
módulos funcionais

4
fases de entrega

11
regras de negócio

22
decisões: 21 fechadas, 1 proposta

MÓDULO A
Ponte com o emulador
fase 1
O canal oficial do emulador, estendido com o mínimo que falta: pausar e velocidade. Identificar o jogo, salvar e recarregar estado, ler memória.

MÓDULO B
Tela e situação
fase 1
Capturar a janela do jogo e transformar em situação: onde o jogador está, o que enfrenta, se está parado, perdido ou dominando.

MÓDULO C
Mestre
fase 1
Observar situação e estado, decidir uma alavanca e um motivo, respeitar o ritmo. Tom e intensidade configuráveis antes da sessão.

MÓDULO D
Alavancas e limites
fase 1
O catálogo universal: pausa, câmera lenta, turbo, estado salvo e recarga. Faixas, intervalo mínimo, reversão da última ação.

MÓDULO E
Sessão e overlay
fase 1
Abrir a sessão com qualquer jogo, mostrar situação, ação e motivo na tela, pausar, retomar, encerrar com resumo.

MÓDULO F
Registro
fase 1
Toda captura, decisão e intervenção gravadas com estado antes, ação, estado depois e motivo. Reproduzível e exportável.

MÓDULO G

Processos automáticos

fase 1

Medir custo e latência, tratar falha do serviço de linguagem e do emulador, manter a versão própria do emulador alinhada ao original.

MÓDULO H

Perfil de memória

fase 2

A camada que muda o mundo, por jogo: inimigo que aparece, recurso que some, item onde não existia. Endereços, faixas, ações de mundo, fontes públicas.

MÓDULO I

Entidades e mundo

fase 3 · mapa de entidades antecipado para a fase 2

Posição de inimigos e personagens: aproximar, afastar, reunir. Exige mapa de entidades por jogo; o mapa (I01) entra na fase 2 com a câmera, porque a cena precisa dos dois.

MÓDULO J

Controle virtual

fase 3

O mestre aciona o controle por um gamepad virtual. Outro tipo de intervenção, com regras próprias.

MÓDULO K

Cena

fase 4 · proposta

A camada que modifica o espaço 3D visto pelo jogador: esconder, duplicar, deformar e recolorir objetos da cena, acrescentar geometria e luz, trocar a aparência de um objeto. Vive dentro do renderizador da versão própria do emulador, em qualquer placa de vídeo, e fala com o mestre pelo mesmo canal. Nunca muda colisão.

Refinamentos vindos do código, registrados aqui: a versão própria do emulador parte da etiqueta corrente do projeto original, não da 2.8.2, porque só ela tem dependências pré-compiladas; a velocidade pode ser lida como "ilimitada" quando o jogador liga pelo atalho do emulador; a confirmação de pausa e velocidade relê o emulador até ver o efeito; morte recente com estado guardado vira aviso explícito ao mestre, ordem no tom implacável; falhas prolongadas pausam o mestre; o fim das ações com duração roda num relógio próprio. Da fase 2: uma foto da memória inteira do jogo pelo canal leva segundos e mistura instantes; a descoberta de endereços pausa o jogo pelo canal, fotografa e retoma; dentro de uma sessão só se lê variável por variável, nunca a memória inteira.

VISÃO

Olho na tela, mão nas alavancas do emulador

O produto é um diretor de jogo por inteligência artificial: ele não joga e não obedece a pedidos do jogador. Ele vê a tela como um espectador, lê o que o emulador sabe da partida e molda o que acontece, como um mestre de mesa que percebe o grupo entediado e aperta o passo, ou percebe o desespero e dá uma trégua. A graça é o jogo deixar de ser o jogo de sempre. Isso acontece em duas camadas: primeiro o ritmo, com as alavancas do próprio emulador, que funcionam em qualquer jogo no primeiro dia; depois o mundo, com o perfil de memória de cada jogo, que é onde o mestre faz aparecer o que nunca aconteceu naquele jogo. O protótipo mostrou que números mudando sem uma situação por trás não fazem um mestre; o que faz é entender a situação e escolher, no momento certo, a intervenção que o jogador sente.

Mestre (serviço de linguagem)

- Recebe a captura da tela e o estado
- Descreve a situação em uma linha
- Decide uma alavanca do catálogo e um motivo
- Pode decidir "nada"
- Não aciona nada; trocável por configuração

Alavancas

- Universais: pausa, câmera lenta, turbo, estado salvo e recarga
- Vêm do emulador, valem para todo jogo
- Cada uma com faixa, duração e intervalo mínimo
- De mundo, por jogo: ações do perfil de memória (fase 2), as que fazem o jogo deixar de ser o comum

Plataforma

- Captura a janela e fala com o emulador pela ponte
- Valida cada ação contra o catálogo
- Relê a situação antes de agir
- Registra tudo com motivo
- Testável sem serviço de linguagem

A PRIMEIRA VERSÃO É

- Um jogador, qualquer jogo de PlayStation 2 que ele possua, nenhum mapeamento prévio.
- Overlay sobre a janela do emulador; produto em português do Brasil.
- O ciclo completo: abrir a sessão, o mestre vê e decide, age pelo emulador, o jogador vê, pausa ou reverte, encerra com resumo.
- Uma versão própria do emulador que só acrescenta ao canal oficial o que falta: pausar e velocidade.
- Toda intervenção registrada e reproduzível.

A PRIMEIRA VERSÃO NÃO É

- Mudar o mundo do jogo: inimigos, recursos, itens (fase 2, por perfil de memória).
- Mover inimigos ou personagens no mapa (fase 3).
- Apertar botões pelo jogador (fase 3).
- Trainer com conversa: o jogador não pede efeito.
- Jogador por inteligência artificial, outro emulador ou outro console.

Por que o ritmo vem antes do mundo. Captura de tela, pausa, velocidade e estado salvo existem para qualquer jogo, sem que ninguém mapeie nada: é o chão, o que garante que o mestre nunca fica mudo. Mudar o mundo exige saber onde ele está na memória, e isso é por jogo, com cobertura pública desigual. Começar pelo ritmo prova a leitura de situação, o ciclo de decisão e o registro com o jogo de sempre; a fase 2 põe sobre isso o que faz o jogo virar outro. A ordem é de engenharia, não de importância.

ATORES

Quem usa e o que fica de fora

Ator é quem inicia ou recebe alguma coisa da plataforma. Na primeira versão, o jogador é o cliente deste projeto: ele aprova cada documento e cada sessão jogada é um teste de aceitação. O mapeador só aparece na fase 2, e a separação existe desde já porque o produto é para outros jogadores também, e a maioria deles só vai jogar.

Jogador

pessoa · ator principal nos módulos C, E

Abre o emulador com o jogo dele, configura tom e intensidade, joga, vê o que o mestre fez, pausa, reverte e encerra.

Mapeador

pessoa · ator principal no módulo H (fase 2)

Cria o perfil de memória de um jogo: variáveis, faixas, ações e proibições. Importa de fontes públicas, valida contra o jogo e publica versões.

Plataforma (rotina)

sistema · ator principal nos módulos A, B, D, F, G

O que o produto faz sozinho: capturar, falar com o emulador, validar, agir, registrar, medir, detectar falha.

Serviço de linguagem

sistema externo · participa de B, C, G

O modelo que lê a tela, descreve a situação e decide alavanca e motivo. Trocável por configuração; nunca guarda estado, nunca aciona o emulador.

Emulador

sistema externo · participa de A, D, G

O PCSX2, em versão própria que só estende o canal oficial de comunicação. Roda o jogo e a BIOS do jogador; o produto não distribui nenhum dos dois.

Fontes de mapas

sistema externo · participa de H (fase 2)

Bases públicas de endereços de memória por jogo, mantidas por comunidades. Cobertura e termos de uso variam por título.

DOIS TIPOS DE ALAVANCA QUE NÃO SE MISTURAM

Universais e por jogo

Alavancas universais

Vêm do emulador e valem para qualquer jogo sem preparo: pausar, câmera lenta, turbo, salvar estado num slot reservado ao mestre e recarregar. São a fase 1 inteira. Nenhuma depende de saber o que está em cada endereço de memória.

Perfil de memória

Por jogo, versionado. Variáveis com significado e faixa, ações de mundo e lista do que nunca se toca. É a fase 2 inteira, começando pelo jogo de abertura: é onde o mestre faz aparecer o que nunca aconteceu naquele jogo. Passa pela mesma validação e pelo mesmo registro das universais.

A sessão viva é o terceiro objeto: o que de fato aconteceu numa partida, captura a captura, decisão a decisão, inclusive "nada". É a fonte de verdade para reverter, retomar e verificar, e é o que separa dirigir de trapacear.

REGRAS DE NEGÓCIO

Onze regras que valem em todos os casos de uso

As cinco primeiras são as regras de mesa: o que mantém a diversão quando o mestre intervém. As demais são regras do produto, e quatro delas nascem de falhas que o protótipo teve de verdade. Cada uma vira teste.

RN-01 **Toda intervenção tem motivo e registro**

Situação, estado antes, ação, estado depois e o motivo do mestre ficam gravados na sessão viva. Intervenção sem registro não existe. Obtemos: mestre auditável.

módulos C, D, F

RN-02 **Dificulta, não derrota**

O mestre pode apertar o ritmo, acelerar, tirar a trégua. Não pode causar derrota direta nem recarregar um estado para fazer o jogador perder o que ganhou. Obtemos: desafio sem roubo.

módulos C, D

RN-03 **Progresso e save são intocáveis**

Cartão de memória e os slots de estado do jogador nunca são tocados. O mestre usa slots reservados a ele, salva antes de qualquer recarga, para que a própria recarga seja reversível, e nunca recarrega sem mostrar. Obtemos: conquista com valor.

módulos A, D

RN-04 **O jogador vê e pode desfazer**

Ação e motivo aparecem no overlay no momento em que acontecem. A última ação pode ser revertida e a sessão pausada a qualquer instante. Obtemos: o controle fica com o jogador.

módulos D, E

RN-05 **Não agir é ação válida**

Há intervalo mínimo entre intervenções e "nada" é uma decisão registrada como qualquer outra. Obtemos: ritmo, não ruído.

módulos C, F

RN-06 **O mestre só age pelo catálogo**

O catálogo é a lista de alavancas universais mais, se houver, as ações do perfil de memória; ele nunca vem do texto do serviço de linguagem, e todo valor fica dentro da faixa declarada. O protótipo somava dinheiro sem teto. Obtemos: nenhuma ação cega.

módulos C, D, H

RN-07 **Serviço de linguagem nunca aciona o emulador**

Ele devolve situação, alavanca e motivo; só a plataforma fala com o emulador, validando contra o catálogo. Obtemos: separação testável sem modelo.

módulos A, C, D

RN-08 **Decidir sobre a situação atual**

A plataforma relê tela e estado antes de agir. Se a situação mudou fora do esperado desde a decisão, a ação é descartada e registrada como tal. O protótipo agia sobre um estado lido segundos antes. Obtemos: nenhuma ação sobre situação velha.

módulos B, D, G

RN-09 **O jogo é do jogador**

Imagem do jogo e BIOS ficam na máquina dele; nunca entram no repositório, nos testes compartilhados nem em qualquer envio. Ao serviço de linguagem vão capturas de tela e o estado resumido, e só isso, com o jogador ciente disso ao abrir a sessão. Obtemos: respeito à propriedade e à licença.

módulos A, B, G

RN-10 **Falha é visível e silencia o mestre**

Serviço fora do ar ou resposta inválida aparece no overlay; sem decisão válida, nenhuma ação acontece; o jogo segue sem o mestre e a sessão retoma. O protótipo ficou meses parecendo vivo com o serviço desligado. Obtemos: nunca parece vivo estando morto.

módulos E, G

RN-11

O que o mestre desenha nunca mente sobre o que mata

Geometria acrescentada pela camada de cena é só visual: não tem colisão e o jogo não sabe dela. Por isso ela nunca esconde inimigo, perigo ou saída, e nunca cria obstáculo falso em caminho obrigatório. Deformar, escurecer e povoar a cena, sim; enganar o jogador sobre o que pode feri-lo, não. Obtemos: "dificulte, nunca derrote" continua valendo com a cena redesenhada.

módulos C, D, K

ORIGEM

O que o protótipo mostrou e por que a direção mudou

Entre fevereiro e março de 2026 o cliente construiu um protótipo sobre Final Fantasy XII no PCSX2: um overlay que lia vida, energia, dinheiro e atributos do personagem direto da memória do emulador e, a cada ciclo, pedia a um serviço de linguagem uma ação de mestre. Vinte scripts, a maioria ferramentas de varredura de memória. Na avaliação do cliente, não funcionou. Este mapa parte dessa conclusão.

FUNCIONOU COMO MECANISMO

- Ler e escrever a memória do jogo de fora do emulador funciona, em tempo real, sem modificar o emulador.
- O ciclo observar, decidir, agir com um serviço de linguagem no meio roda e devolve decisões com motivo.
- Um overlay sobre a janela do jogo é suficiente como interface: estado, última ação, motivo, histórico.

NÃO FUNCIONOU COMO EXPERIÊNCIA NEM COMO ENGENHARIA

- Com só números na entrada, o mestre decidia o que meia dúzia de regras fixas decidiriam; o jogador via vida e dinheiro mudando, não um evento. É o motivo desta revisão.
- A base da memória era procurada por varredura do valor do dinheiro, com doze scripts para um problema que o canal oficial do emulador resolve.
- Posições de entidades caçadas por diferença de movimento: trinta candidatas, duas plausíveis, nenhuma confiável.
- O serviço de linguagem usado foi descontinuado e o código engolia o erro; endereços de uma sessão e a chave do serviço ficaram gravados no código.

QUATRO LIÇÕES QUE VIRARAM REGRA

Falha silenciosa
A exceção do serviço virava a decisão "nada" com o erro escondido no motivo. Agora é RN-10: falha visível, mestre em silêncio, nenhuma ação.

Estado velho
O estado era lido, o serviço demorava, e a escrita usava o valor antigo. Agora é RN-08: reler antes de agir, descartar se mudou.

Sem teto
Dar dinheiro somava sem limite; atributos tinham tetos fixos no código. Agora é RN-06: catálogo com faixa, nunca vindo do texto do serviço.

Ação dentro do serviço
O serviço escolhia entre ações cujas escritas estavam coladas no mesmo código. Agora é RN-07: o serviço devolve intenção, a plataforma executa e valida.

O que se aproveita. O desenho do overlay e o ciclo de decisão. O que se descarta: toda a varredura de memória, os endereços absolutos, o mapeamento de entidades, a chave gravada no código e a tabela de deslocamentos do personagem, que valia só para uma versão do jogo que o cliente não tem mais.

ROTEIRO DE IMPLEMENTAÇÃO

Três documentos, um portão entre cada, depois código

O projeto é conduzido como processo de engenharia, do protótipo ao produto. Nenhuma etapa começa sem a anterior aprovada pelo cliente. O que for aprendido numa etapa volta para o documento anterior como revisão numerada, não como conversa perdida.

1. Mapa do projeto
Este documento: visão, atores, módulos, regras, decisões e lacunas.

→ **Portão**
Aprovação do cliente.

→ **2. Casos de uso**
Uma ficha por caso: objetivo, fluxo, alternativas, exceções, resultado, regras. Entregue módulo a módulo.

→ **Portão**
Aprovação do cliente.

3. Blueprint técnico
Arquitetura, extensão do canal do emulador, protocolo entre mestre e plataforma, cards de implementação com critério de aceite.

→ **Portão**
Aprovação do cliente.

→ **4. Código, card a card**
Branch por card, revisão, integração contínua verde, critério de aceite provado.

→ **Portão por fase**
Sessões jogadas pelo cliente.

COMO CADA CARD É FECHADO

- Critério de aceite escrito antes do código, no blueprint.
- Plataforma testada isolada, sem serviço de linguagem, contra um emulador simulado que responde ao mesmo canal.
- Mestre testado numa bancada de capturas e estados sintéticos em que toda decisão tem de cumprir as regras de negócio.
- Integração contínua verde é o registro de entrega.
- Mudança de requisito entra como caso de uso novo, com código próprio.

COMO A QUALIDADE É MEDIDA

- Registro de defeitos: cada travada ou intervenção estranha em sessão jogada vira item com causa.
- Medir antes de corrigir: decisão, custo e latência têm número antes de qualquer ajuste.
- Revisão pós-fase: o que funcionou, o que falhou, o que muda no processo.
- Lista de lacunas fechada e republicada no fim de cada fase.

O ESQUELETO AMBULANTE

O primeiro código entregue é a fatia mais fina de ponta a ponta: a versão própria do emulador com pausar e velocidade acrescentados ao canal oficial, a captura da janela, o mestre com quatro alavancas e "nada", o overlay e o registro, rodando em qualquer jogo que o cliente tenha. Ele prova a arquitetura antes de qualquer funcionalidade ganhar corpo, e é o primeiro item que o cliente joga. Emulador, BIOS e jogo já rodam na máquina do cliente; o que o destrava é o ambiente de compilação do emulador (lacuna LC-10).

FASES DE ENTREGA

O que entra em cada fase

A fase 1 é o mestre jogável em qualquer jogo, com tela e alavancas do emulador: o ritmo. A fase 2 é o mundo: o perfil de memória do jogo de abertura, com as ações que fazem o jogo deixar de ser o comum, e depois o produto para outros jogadores. A fase 3 dá ao mestre as mãos: entidades e controle. A fase 4, proposta nesta revisão, dá ao mestre a cena: o espaço 3D visto pelo jogador passa a ser modificável em tempo real, dentro do próprio emulador.

Fase 1 · mestre jogável em qualquer jogo

módulos A a G

Abrir um jogo no emulador, abrir a sessão, jogar com o mestre vendo a tela e agindo pelas alavancas do emulador dentro das regras de negócio, ver, pausar, reverter, encerrar. Termina quando o cliente joga uma sessão inteira com toda intervenção registrada e nenhuma fora do catálogo.

- A Canal oficial estendido: pausar e velocidade
- A Identificar o jogo
- A Estado salvo em slot reservado
- B Captura da janela
- B Situação em uma linha
- C Observar e decidir
- C Tom e intensidade
- D Catálogo universal com faixas
- D Reverter e pausar
- E Overlay
- E Resumo da sessão
- F Registro reproduzível
- G Falha visível
- G Custo e latência

Fase 2 · o mundo muda: perfil de memória e outros jogadores

módulo H e instalação

Primeiro o perfil do jogo de abertura, com ações de mundo: recurso que some ou aparece, inimigo que vem, estado que muda. Criado à mão, importado de fontes públicas ou com descoberta assistida, validado contra o jogo em execução, entrando no mesmo catálogo e no mesmo registro. Depois, perfis compartilháveis e instalação para outros jogadores, só com a validação da lacuna LC-08 feita. **Ordem revisada em 27/10 (DC-22):** a primeira metade começa agora, adiantada pela decisão do cliente de fazer a engenharia reversa do jogo, e começa pela câmera e pelo mapa de entidades (I01), que são o que a camada de cena precisa; os cards de robustez da fase 1 que faltam (retomar sessão, custo com teto, exportar) e a sessão de aceite da fase 1 ficam para depois dessa primeira metade.

- H Perfil do jogo de abertura com ações de mundo
- H Importar de fontes públicas
- H Descoberta assistida
- H Publicar e importar perfil
- – Instalação para outros jogadores

Fase 3 · entidades e controle

módulos I, J

Mover inimigos e personagens no mapa, para jogos com mapa de entidades, e acionar o controle por um gamepad virtual. São outros tipos de intervenção, com regras próprias, e só entram sobre registro e catálogo já provados. As regras de negócio continuam valendo: tudo visível, tudo registrado, nada de derrota direta.

- I Posição de entidades
- I Aproximar, afastar, reunir
- J Controle virtual
- J Regras próprias de intervenção por controle

Fase 4 · a cena muda: camada de cena no emulador

módulo K · proposta

O renderizador do emulador vê cada triângulo que o jogo manda desenhar, já projetado na tela, com a textura que o veste e a profundidade de cada ponto. A camada de cena se apoia nisso em três níveis, cada um medido antes do seguinte. **Nível 1, sem câmera:** identificar objetos pela textura e pela forma, escondê-los, duplicá-los, deformá-los, recolori-los; trocar textura o emulador já faz. **Nível 2, com a câmera lida da memória do jogo (fase 2):** acrescentar geometria e luz no mundo, com profundidade certa, ocultada pelo que está na frente. **Nível 3:** trocar a aparência inteira de um objeto, o que exige saber também onde ele está. Em nenhum nível a colisão muda: o jogo continua decidindo por onde se anda e o que fere (RN-11). O caminho sem ray tracing e sem placa específica foi escolha do cliente (DC-20), sabendo que é a parte mais ambiciosa do projeto e a mais cara de acompanhar no original.

- K Prova medida: esconder um objeto e desenhar outro com profundidade certa
- K Identidade de objeto entre quadros
- K Câmera por jogo, vinda do perfil de memória
- K Alavancas de cena no catálogo, com as mesmas regras
- K Comandos de cena no canal

Jogo de abertura. Resident Evil 4, versão americana, já rodando no emulador do cliente: tensão constante e tela fácil de ler para o mestre. Com alavancas universais, qualquer outro jogo dele também serve na fase 1 sem nada a refazer; o perfil de memória da fase 2 começa por este. Para testes automáticos e integração contínua, um emulador simulado que responde ao mesmo canal, mais um banco de capturas de tela gravadas, para que plataforma e mestre rodem sem emulador, sem jogo e sem serviço de linguagem.

DECISÕES

Vinte e três decisões: vinte e duas fechadas, uma proposta

As oito abaixo foram respondidas ao longo da conversa em 26 de outubro de 2026; as seis seguintes foram propostas por este documento e aprovadas; as cinco últimas nasceram nos Casos de Uso e fecharam no Blueprint Técnico. DC-20 e DC-21 foram propostas na revisão 6 e aprovadas no mesmo dia; DC-22, da revisão 7, foi aprovada e executada; DC-23 é a proposta desta revisão. Mudança posterior entra como revisão numerada, com data, e propaga para os casos de uso e para o blueprint.

CÓDIGO	DECISÃO	ESCOLHA	POR QUÊ	ESTADO
DC-01	Usuário final	Outros jogadores também, não só o cliente	Define desde já perfis compartilháveis e a separação entre quem mapeia e quem joga.	fechada 26/10
DC-02	Ordem das camadas	Fase 1 é o ritmo: tela e alavancas do emulador. Fase 2 é o mundo: perfil de memória, começando pelo jogo de abertura. Entidades e controle na fase 3	O ritmo vale para qualquer jogo no primeiro dia e prova o ciclo; o mundo é o que faz o jogo deixar de ser o comum, e vem logo depois. Substitui a escolha anterior, só valores numéricos.	revisada 26/10
DC-03	Visibilidade	Transparente: overlay com ação e motivo, pausa e reversão	O jogador mantém o controle; é o que torna o mestre aceitável em vez de intrusivo.	fechada 26/10
DC-05	Alvo	Qualquer jogo de PlayStation 2 no PCSX2; o PlayStation 2 é o objetivo, não o meio	Escolha do cliente, ciente de que um jogo de computador com modding daria mais alavancas.	fechada 26/10
DC-06	Ponte com o emulador	O canal oficial de comunicação do emulador; varredura de memória por valor nunca mais	Já oferece leitura e escrita, estado salvo, identificação do jogo e status; elimina os doze scripts do protótipo.	fechada 26/10
DC-07	Emulador	Só PCSX2; outros emuladores e consoles fora	Um emulador, um canal, uma versão própria para manter.	fechada 26/10
DC-13	Versão própria do emulador	Versão própria do emulador com o mínimo por recurso: na fase 1, só o servidor do canal; na fase 4, também o renderizador, onde a camada de cena mora	Revisada em 27/10 pela fase 4: o renderizador é a parte que mais muda no projeto original, então acompanhar as versões novas fica mais caro (segunda metade de LC-10). Cada mudança continua isolada e nomeada. Revisada de novo no mesmo dia: a versão própria é um projeto nosso e não será proposta ao original; o projeto original tem regras próprias de contribuição que não cabem aqui.	revisada 27/10 (2x)
DC-14	Fronteira de licença	O emulador modificado é distribuído com o código, na mesma licença do original; o mestre é programa separado, que fala com o emulador por rede local, com licença própria	Decisão de desenho tomada de propósito: o que estende o emulador herda a licença dele; o que fica fora não. A leitura detalhada da licença é tarefa do blueprint.	fechada 26/10

DECISÕES · CONTINUAÇÃO

Seis propostas da revisão 3, cinco vindas do Blueprint e três das revisões 6 e 7

As seis primeiras aprovadas sem alteração na revisão 3; o nome continua provisório. As cinco seguintes foram candidatas nos Casos de Uso e fecharam com a aprovação do Blueprint Técnico, onde estão detalhadas. DC-20 e DC-21, da revisão 6, foram aprovadas e a fase 4 desceu para os Casos de Uso e para o Blueprint. DC-22, desta revisão, está na tabela anterior e aguarda a aprovação do cliente; com ela, a fase 2 desce com o módulo H e o caso I01 detalhados.

CÓDIGO	DECISÃO	ESCOLHA	POR QUÊ	ESTADO
DC-04	Papel da inteligência artificial	Mestre invisível que molda a partida	Não é jogador por inteligência artificial nem trainer com conversa; é o produto do protótipo.	fechada 26/10
DC-08	O jogo é do jogador	O produto não distribui jogo nem BIOS; imagem de jogo nunca entra no repositório	Propriedade e licença são do jogador; o produto só vê a tela e fala com o emulador.	fechada 26/10
DC-09	O jogador não pede ação	Configura tom e intensidade antes da sessão; durante, só pausa, reverte e encerra	Se o jogador escolhe o efeito, é trainer. Transparência com pausa e reversão é o limite que mantém o mestre.	fechada 26/10
DC-10	Idioma	Produto em português do Brasil; rótulos de perfil podem manter o nome original do jogo	Público do cliente; o nome original evita ambiguidade no mapeamento.	fechada 26/10
DC-11	Nome, pasta e repositório	"IAEmPS2" (provisório); pasta Projetos-SSD\iaemps2; repositório público, branch e merge, nunca commit direto na principal	Mesmo esquema de trabalho dos outros projetos. Público porque a integração contínua em repositório privado é cobrada e travou no primeiro card; jogo, BIOS, capturas e sessões nunca entram nele (RN-09).	revisada 26/10
DC-12	Origem do perfil de memória	À mão, importado de fonte pública ou descoberta assistida; qualquer origem passa por validação contra o jogo em execução antes de entrar numa sessão	Perfil errado age no lugar errado; a validação é a mesma independente da origem.	fechada 26/10
DC-15	Slots de estado do mestre	Slots 90 a 99, fila de 10 por jogo, o mais antigo sobrescrito	Provado no emulador do cliente: o salvamento funciona e a limpeza do emulador só alcança os slots 1 a 10.	fechada 26/10
DC-16	Formato da situação	JSON fechado: descrição em uma linha, fase numa lista fixa, grau de aperto de 0 a 4, confiança	Campo fora da lista é resposta inválida; é o que torna a leitura de tela testável.	fechada 26/10
DC-17	Tons e intensidades	Tons protetor, imprevisível (padrão) e implacável; intensidades leve, média (padrão) e forte, que fixam intervalo mínimo e faixas	O jogador configura o comportamento sem nunca escolher uma ação.	fechada 26/10
DC-18	Gestos do jogador	Ctrl+Alt+Z reverter, Ctrl+Alt+P pausar e retomar o mestre, Ctrl+Alt+Q encerrar, mais botões no overlay	Combinações que o emulador não usa; colisão com o jogo é conferida no esqueleto.	fechada 26/10
DC-19	Aviso do que sai da máquina	Texto versionado, aceite registrado com data, reapresentado ao trocar de serviço	Capturas de tela saem da máquina; o jogador tem de saber antes da primeira.	fechada 26/10

CÓDIGO	DECISÃO	ESCOLHA	POR QUÊ	ESTADO
DC-20	Como o mestre modifica o espaço 3D	Camada de cena própria dentro do renderizador da versão própria do emulador: identifica, esconde, deforma e acrescenta geometria na tela, em qualquer placa de vídeo, sem ray tracing e sem plataforma de terceiros	Escolha do cliente em 27/10 entre quatro caminhos: alugar máquina com placa NVIDIA para medir a integração de ray tracing que o fabricante mostrou sobre o emulador; esta camada própria; só memória do jogo (fase 2); trocar o alvo por um motor aberto. A camada própria roda na máquina que o cliente tem e mantém o PlayStation 2 como objetivo (DC-05). O que já existe foi levantado: a integração de ray tracing exige placa específica e é cru; a captura de cena do emulador para arquivo existe, mas sem câmera.	fechada 27/10
DC-21	Ordem da fase 4	Uma prova medida já (nível 1, sem câmera) para saber se a camada é viável; a fase 4 inteira só depois da fase 2, de onde vem a câmera	Os triângulos chegam ao renderizador já projetados; para pôr geometria nova no mundo é preciso a câmera, que mora na memória do jogo e é a fase 2. Medir antes de investir é a regra do projeto (LC-10 foi assim).	fechada 27/10
DC-22	Ordem da fase 2	Fase 2 adiantada: começa agora pela câmera e pelo mapa de entidades do jogo de abertura (I01 vem da fase 3 para cá); retomar sessão, custo com teto, exportar e a sessão de aceite da fase 1 ficam para depois	A prova medida da cena mostrou três vezes que o desenho não diz quem é quem (forma muda com a câmera, pele e cabelo iguais para jogador e inimigo, material de inimigo veste o cenário). A posição e a câmera moram na memória do jogo. O cliente decidiu em 27/10: "se não der certo, engenharia reversa do jogo". Memória é o primeiro dos três níveis dessa engenharia (depois código, depois formatos).	fechada 27/10
DC-23	Mapa completo antes das ações de mundo	Mapear o jogo de abertura inteiro de forma sistemática (todas as funções decompiladas, inclusive os módulos da ISO realocados pelo próprio mestre; estruturas; catálogo de capacidades; roteiro por jogo), como etapa S9, antes de pôr ações de mundo no catálogo	A engenharia reversa sob demanda, uma função por pergunta, custou um ciclo por pergunta e deixou buracos: a ligação de módulos do jogo foi usada antes de ser lida e travou o jogo três vezes. O cliente decidiu em 29/10: "acho bem mais válido que fazer sob demanda". O mapa é pago uma vez e consultado; o que já funcionou (vigia, chamada remota, carga de módulos) vira roteiro para o próximo jogo.	proposta 29/10

O que ficava para o blueprint foi decidido nele: linguagem e estrutura do repositório, tamanho da extensão do canal, ciclo de captura e decisão, formato da sessão viva. A escolha do serviço de linguagem continua por medição, na bancada.

LACUNAS

O que ainda não está resolvido

Lista republicada no fim de cada fase. Oito fechadas: três na máquina do cliente, duas no Blueprint, três por medição. LC-17 (nível 3) ganhou caminho concreto na revisão 8; LC-18 aberta na mesma revisão.

Como usar este documento no dia a dia. Ele é a origem de tudo: os casos de uso detalham os módulos daqui, o blueprint implementa os casos de uso, e cada card de código aponta para o caso de uso que cumpre. Quando uma necessidade nova surgir, ela entra primeiro aqui, como módulo, regra, decisão ou lacuna com código próprio, e só depois desce. Assim o escopo continua visível e negociável, e nenhuma decisão fica só na conversa.

CÓDIGO	LACUNA	ONDE RESOLVE	RISCO SE FICAR ABERTA
LC-01 fechada 26/10	Jogo e BIOS na máquina do cliente. Emulador, BIOS e Resident Evil 4 (SLUS-21134, versão 1.01) instalados e verificados pelo canal. É o jogo de abertura.	Verificação feita nesta revisão	Nenhum.
LC-02 fechada 26/10	O emulador instalado expõe a memória pela interface oficial. Verificado no executável 2.8.2: símbolos exportados e servidor do canal embutido.	Verificação feita nesta revisão	Nenhum.
LC-03 fechada 26/10	Identificar o jogo em execução: o canal devolve título, serial, soma de verificação e versão por comando próprio.	Código-fonte da versão 2.8.2	Nenhum.
LC-04	Cobertura real e termos de uso das fontes públicas de mapas, jogo a jogo. Para o jogo de abertura há listas públicas de endereços (vida, munição, personagem), uma delas conferida ao vivo em 27/10; termos de uso não verificados.	Casos de uso do módulo H (H02); levantamento na fase 2	A camada de memória vale para poucos jogos.
LC-05	Como medir a qualidade do mestre sem gabarito: bancada de situações sintéticas em que toda decisão cumpre as regras, mais sessões jogadas; a diversão continua julgamento do jogador.	Card S6-01 do Blueprint	Sem medida, regressão passa despercebida.
LC-06	Custo e latência por sessão com captura de tela a cada ciclo, e limite aceitável para o cliente. Imagem custa mais que texto.	Medição no esqueleto ambulante	Produto inviável de operar.
LC-07 fechada 27/10 para a cena	Levantamento do que já existe. Para modificar o espaço 3D: integração de ray tracing do fabricante de placas sobre um fork do emulador, mostrada em feira em 2026, que reconstrói a cena e troca malhas e materiais, mas exige placa específica e está crua; captura de cena do emulador para arquivo, sem câmera; nos outros emuladores, só troca de texturas. Para alterar jogos por evento externo (fase 2), o levantamento continua pendente.	Feito nesta revisão; fase 2 antes de começar	Construir o que já existe.
LC-08	Validação com outros jogadores antes da fase 2: medir interesse e qual jogo pedem primeiro.	Fim da fase 1	Produto certo para uma pessoa, errado para o resto.
LC-09	Nome definitivo do produto.	Decisão do cliente, a qualquer momento	Nenhum.
LC-10 fechada 26/10	Ambiente de compilação do emulador. Medido na máquina do cliente: compilação completa em 7 minutos, recompilação após mudar o servidor do canal em 3 minutos, 2,4 GB de objetos, nada a instalar, zero erros. O custo de acompanhar o original cresce com a fase 4 (DC-13 revisada).	Card S4-01 do Blueprint, medição gravada no repositório	Nenhum para a fase 1; reavaliar na fase 4.

CÓDIGO	LACUNA	ONDE RESOLVE	RISCO SE FICAR ABERTA
LC-11 fechada 26/10	Confirmação de salvar e recarregar estado: o emulador executa de forma assíncrona e reporta erro só na tela. Fechada por medição: confirmação pelo arquivo com tamanho estável, 0,9 s no jogo de abertura.	Blueprint, capítulo 6	Nenhum.
LC-12 fechada 26/10	Volta da velocidade ao normal se a plataforma cair. Fechada por desenho: vigia dentro do servidor do canal do emulador, com duração no próprio comando.	Blueprint, capítulo 6	Nenhum.
LC-13	Captura com o jogo em tela cheia exclusiva. A captura pela janela foi provada em modo janela; falta provar em tela cheia sem borda.	Card S2-01 do Blueprint	Jogador em tela cheia fica sem mestre.
LC-14	De onde vem a câmera. Os triângulos chegam ao renderizador já projetados na tela; a matriz da câmera mora na memória de cada jogo, em lugar diferente por jogo. Sem ela, só o nível 1 da cena existe.	Perfil de memória do jogo de abertura (fase 2), com o campo "câmera"	Fase 4 para no nível 1: esconder e deformar, nunca acrescentar.
LC-15 fechada 27/10	Identidade estável de um objeto entre quadros. Fechada por medição, pela negativa: não vem do desenho. A forma do lote muda com a câmera (recorte por quadro), pele e cabelo são o mesmo material para jogador e inimigos, e materiais de inimigo vestem o cenário. Vem da memória do jogo: posição da entidade projetada pela câmera (fase 2, DC-22).	Cards S7-00 e S7-01 do Blueprint, medição gravada no repositório	Nenhum: a cena mira pela memória.
LC-16	Convenção de profundidade por jogo. A profundidade que o emulador recebe tem escala e sentido que variam por jogo; geometria acrescentada com profundidade errada aparece na frente do que deveria tapá-la.	Prova medida da fase 4, depois o perfil de memória	Cena acrescentada parece colada na tela, não no mundo.
LC-17	Nível 3 da engenharia reversa: como o jogo de abertura guarda modelos, colisão e fases, para mudar geometria de verdade, com colisão. Meta declarada pelo cliente em 27/10. **Caminho concreto desde 29/10:** um personagem é um módulo de código realocável (.rel, a IA) mais um pacote de modelo (.dat, com a malha .bin e as texturas .tpl); o mestre já carrega os dois e os combina; um modelo novo exige destrinchar o .bin e o .tpl e escrever um conversor, reaproveitando a IA de um módulo existente. **Medido em 30/10 e fechado em 10/11** (S9-02-colisao-e-terreno): chão andável gerado do modelo visual da sala e registrado como colisão, provado com o cliente no controle; verificação automática sem o cliente (10 459 passos). Lacunas que voltam nesta fase: travessia de parede em 1,4 % dos passos e salto nativo disponível em 10 de 30 muros gerados. O acabamento foi interrompido em 10/11 por decisão do cliente, para não repetir o sob demanda que a DC-23 proíbe.	Depois do S9 (mapa completo); entra como fase própria	A cena só muda na aparência; o espaço continua o do jogo.
LC-18	Ada de cena na campanha do Leon. O módulo dela (cSubAda) não tem pacote de modelo próprio; criada com o modelo da Ada jogável, nasce e anda, e o jogo trava na colisão segundos depois. O sistema de cenas (evm, SceAt*) prepara colisão e animação de outro modo, ainda não lido.	S9-02 (sistema de cenas nas estruturas)	Só personagens com módulo e modelo próprios podem ser criados; os de cena, não.